

Mathematical and Structural Architecture of a C++ Interpreter

Bridging Graph Theory, Formal Logic, and Software Engineering

Interpreter Development Team

April 17, 2026

Table of Contents

- 1 Introduction & Architecture
- 2 Lexical Analysis & Tokenization
- 3 Formal Grammar & Syntactic Analysis
- 4 System Modeling with Abstract Syntax Trees (AST)
- 5 Dynamic State & Memory Management
- 6 Tree-Walk Evaluation

The Engineering Challenge

Translating an unstructured, continuous sequence of human-readable characters into a rigorously defined, deterministic mathematical execution framework.

Intersection of Disciplines:

- **Mathematics:** Directed Acyclic Graphs (DAGs), state mapping sequences, Boolean logic.
- **Software Engineering:** Recursive architectures, polymorphism, manual memory management, hash-based memory representations.

High-Level System Architecture

The system maps to a standard pipeline utilizing several disjoint but inherently cooperative phases.

- **Scanner:** Discretization phase.
- **Parser:** Structural topology generation Phase.
- **Evaluator:** Graph reduction and Execution phase.
- **Environment:** Context persistence mapping.

The Pipeline Model

Execution Pipeline Flow

Raw Source $\xrightarrow{\text{Lexer}}$ Tokens $\xrightarrow{\text{Parser}}$ AST $\xrightarrow{\text{Evaluator}}$ Result/Side-effect

- Characters shift mathematically to discrete vectors.
- Vectors shift to syntax trees representing precedence.
- Syntax trees undergo recursive post-order traversals to derive states.

Lexical Tokenization: Concept

Continuous-to-Discrete Extraction

Lexical analysis reads a stream of characters from memory and converts it into a sequence of discretized objects (*Tokens*).

This is highly analogous to continuous signal sampling in digital systems or the preprocessing embedding phase in NLP architectures. We convert raw topological manifolds (strings) into state-distinct integers and classified sequences.

Modeling as a Deterministic Finite Automaton (DFA)

- The Scanner models a DFA iteratively processing inputs.
- Transitions evaluate the structural validity of consecutive ASCII bounds.
- **States:** Start, In-String, In-Number, In-Identifier, Done.
- **Output:** Vectors classified functionally for the parser domain.

The Token Structure

The Token struct is the primary mathematical node at the lexical level.

```
1 enum class TokenType {  
2     LEFT_PAREN, RIGHT_PAREN, PLUS, MINUS, NUMBER, STRING, EOF_TOKEN // etc...  
3 };  
4  
5 struct Token {  
6     TokenType type;  
7     string lexeme;  
8     string literal; // Retains the scalar evaluation natively  
9 };  
10
```

Each token inherently binds the lexical type with its original planar character sequence (lexeme).

Scanner C++ Implementation: Core Logic

We use iterative advancement over string matrices.

```
1 char advance() {  
2     return source[pos++];  
3 }  
4  
5 char peek() const {  
6     return isAtEnd() ? '\0' : source[pos];  
7 }  
8  
9 bool match(char expected) {  
10     if (isAtEnd() || source[pos] != expected) return false;  
11     pos++;  
12     return true;  
13 }  
14
```

Handling Maximal Munch & Strings

To enforce token uniqueness, the state machine implements *Maximal Munch* – eating as many characters as grammatically allowed to form the largest distinct unit.

```
1 void scanString() {  
2     string value;  
3     while (!isAtEnd() && peek() != '"') {  
4         if (peek() == '\n') line++;  
5         value += advance();  
6     }  
7     advance(); // consume closing "  
8     addToken(TokenType::STRING, "\"" + value + "\"", value);  
9 }  
10
```

Context-Free Grammar (CFG) via BNF

The formal structural constraints are defined via Backus-Naur Form (BNF).

Mathematical Rules

expression $\rightarrow$ *assignment*

assignment $\rightarrow$ IDENTIFIER " = " *assignment* | *logic_or*

logic_or $\rightarrow$ *logic_and* (" or " *logic_and*)^{*}

logic_and $\rightarrow$ *equality* (" and " *equality*)^{*}

This recursively ensures that lower-precedence actions derive higher-precedence nodes.

Transitioning from Tokens to ASTs

Once tokens are formulated, linear context falls short of handling strict structural boundaries. Processing tokens sequentially inherently violates computational order-of-operation axioms.

Solution: Generate a Directed Acyclic Graph (DAG) known as the *Abstract Syntax Tree*. Each node governs a hierarchy evaluated structurally bottom-up.

Operator Precedence

Rule	Operators	Associativity
Primary	(), literals	N/A
Unary	!, -	Right-to-left
Factor	*, /	Left-to-right
Term	+, -	Left-to-right
Comparison	<, <=	Left-to-right
Equality	==, !=	Left-to-right

The grammar implicitly hard-codes execution precedence into the tree depth.

Left-Associative Trees

For commutative and non-commutative linear transformations (like $5 - 3 - 1$), the parser enforces left-association.

This means our generation loops group earlier terms into nested left-heavy binary trees.

Recursive Descent Parsing

Our translation relies on a Top-Down Recursive Descent Parser. Each context-free grammar production rule accurately corresponds to a distinct C++ function.

- Evaluates the highest-priority derivation first (the lowest precedence operation).
- Recursively descends down to terminal nodes (Primitives).
- Consumes logic strictly in $O(N)$ execution time.

Recursive Descent: Code Implementation

A clear demonstration of left-heavy recursion to form mathematical groups:

```
1 Expr* equality() {  
2     Expr* left = comparison();  
3     Token op;  
4     while (match({TokenType::EQUAL_EQUAL, TokenType::BANG_EQUAL})) {  
5         op = previous();  
6         Expr* right = comparison();  
7         left = new Binary(left, op, right);  
8     }  
9     return left;  
10 }  
11
```

The Expr & Stmt Architectural Hierarchies

Operations strictly divide into two independent categories forming mathematical graphs.

- **Expr (Expression):** Directed vertices determining an evaluation computation reducing strictly to a pure scalar state.
- **Stmt (Statement):** Top-level commands enforcing spatial or temporal side-effects upon the persistent state map without propagating values back up.

Polymorphism for Node Representation

Because nodes consist of distinct variables (Unary, Binary, Literal), the system employs C++ Object-Oriented Polymorphism.

```
1 struct Expr {  
2     // Top-Level AST Node interface  
3     virtual ~Expr() = default;  
4     virtual string toString() = 0;  
5     virtual LoxValue evaluate() = 0;  
6 };  
7
```

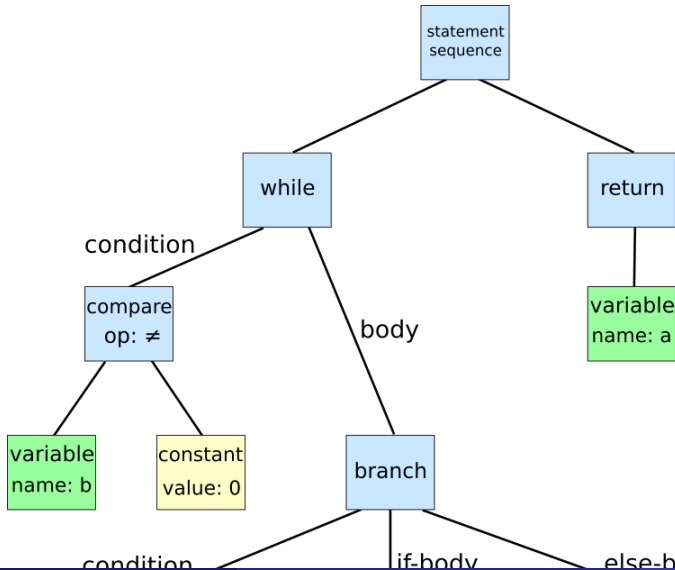
All evaluations naturally follow Virtual Function Tables (v-tables) resolving exact types elegantly dynamically at runtime.

Binary Node Structural Example

The Binary AST node effectively holds mathematical operations combining two edges.

```
1 struct Binary : Expr {
2     Expr* left;      // Left child branch
3     Token op;        // The central scalar operator
4     Expr* right;     // Right child branch
5
6     Binary(Expr* left, Token op, Expr* right)
7         : left(left), op(op), right(right) {}
8
9     LoxValue evaluate() override {
10         // Post-Order reduction methodology
11     }
12 };
13
```

Visualizing the Directed Acyclic Graph (AST)



Handling Typed Scalars in C++ (std::variant)

To strictly adhere to dynamic type safety in a highly explicit, rigid type system (C++), scalar node reductions are bounded mathematically via 'std::variant':

Variant Definition

```
using LoxValue = std::variant<std::monostate, double, bool, std::string,  
...>
```

This resolves the halting problem constraint of storing dynamic types in a typed vector.

The Environment as a Mapping Function

A continuous computational phase cannot exist without variable retention maps.

The E Tensor

We define the context mapping as: $E : \text{String} \rightarrow \text{Value}$

This acts fundamentally as the system's "Hidden State Tensor". With each subsequent time-step (statement), the Environment captures or overwrites explicit definitions mirroring RNN behavior.

C++ Implementation: Hash Maps

Spatial lookup bounds require strict $O(1)$ amortized memory retrieval functions. Thus, this maps mathematically perfectly to 'std::unordered_map'.

```
1 class Enviroment {  
2 public:  
3     Enviroment* par; // Pointer to parent state (chained scoping)  
4     unordered_map<string, LoxValue> var; // The memory grid  
5  
6     // ...  
7 };  
8
```

Lexical Scoping and Memory Chaining

Scoping limits mathematical causality. Variables defined in depth nodes cannot bleed linearly outward.

- Local matrices point recursively backward to global arrays via pointers.
- Any map traversal that hits NULL causes an intentional halting fault.

Variable Definition vs. Assignment

There is a strict logical derivation between establishing a map axis and overwriting scalar variables.

```
1 void define(Token key, LoxValue val) {  
2     var[key.lexeme] = val; // Always creates bounded domain  
3 }  
4  
5 void assign(Token key, LoxValue val) {  
6     Enviroment* curr = this;  
7     while(curr != NULL && curr->var.find(key.lexeme) == curr->var.end()) {  
8         curr = curr->par; // Traverse recursive parents  
9     }  
10    if(curr == NULL) exit(70); // Runtime exception  
11    curr->var[key.lexeme] = val; // Overwrite  
12 }  
13
```

The Importance of State Distinctiveness

If a variable is tracked via *assignment* prior to *declaration*, it signifies a time reversal paradox in formal causal logic computation. The mapping architecture enforces the strict linear timeline of the interpreter via this implementation.

The Execution Model

Execution shifts entirely to a Post-Order Traversal sequence across the generated syntax graph.

- **Recurse Left** → Solve node.
- **Recurse Right** → Solve node.
- **Collapse Parent** → Merge sub-branches applying scalar operations.

Expression Reductions (Returning Values)

Expressions collapse upward cleanly.

```
1 LoxValue evaluate() override {  
2     LoxValue l = left->evaluate();  
3     LoxValue r = right->evaluate();  
4  
5     switch (op.type) {  
6         case TokenType::PLUS:  
7             if((holds_alternative<double>(l) && holds_alternative<double>(r)))  
8                 return getD(l) + getD(r); // Pure scalar mathematical sum  
9             // ...  
10    }  
11 }  
12
```

Statement Executions (Side Effects)

Statements execute transversally with no upward scalar inheritance, modifying global mapping limits inline.

```
1 class ReturnStmt : public Stmt{
2 public:
3     Expr* value;
4
5     void execute() override{
6         LoxValue val = value != NULL ? value->evaluate() : nil{};
7         returnException rval(val);
8         throw(rval); // Interrupting procedural execution to jump causality cleanly
9     }
10 };
11
```

Handling Abstract Tree Dynamics (Control Flow)

Loops and logical jumps (e.g. `while`, `if`) are not linear vectors.

- They behave as recursive re-evaluation subroutines inside the C++ host boundary.
- Traversal pauses on child evaluations based entirely on mapping state validation rules (*Truthiness*).

Mathematical Bounds

A tree graph collapsing with incorrectly matched scalars (e.g. division by zero, type mismatches natively) throws direct memory interrupt violations.

Conclusion: By leveraging rigorous graph topology mapping, formal context free grammars, and continuous tensor environments, we enforce deterministic execution paths mathematically bound securely inside modern Software Engineering C++ paradigms.

Questions?

End of Presentation